

ADR 004 - Adoção dos padrões Strategy, Factory e Template Method

Data: 11/08/2026 (Entrega) | **Status:** Aceito

Contexto

Tanto o ms-checkout quanto o ms-pagamento precisam tratar diferentes meios de pagamento (Pix, cartão e, no ms-pagamento, também boleto) com regras de processamento e validação distintas para cada tipo, escolhidas dinamicamente a partir do campo tipo recebido no corpo da requisição. Ao mesmo tempo, o fluxo geral do checkout (autenticar o cliente, processar o débito, atualizar o status do pedido e emitir o recibo) é sempre o mesmo, independentemente do meio de pagamento escolhido.

Sem alguma forma de abstração, essa variação tenderia a se materializar como blocos if/else ou switch dentro dos controllers ou serviços, repetidos em mais de um lugar (validação, processamento, log de etapas), o que fere diretamente os princípios de alta coesão e baixo acoplamento definidos como critério de avaliação do projeto (ADR 001).

Decisão

Adotar três padrões GoF combinados, cada um resolvendo um eixo diferente dessa variação:

1. **Strategy:** as interfaces PaymentStrategy (ms-checkout) e ValidacaoStrategy (ms-pagamento) definem um contrato comum de processamento/validação de pagamento. Cada meio de pagamento tem sua própria implementação concreta (CardStrategy, PixStrategy no checkout; CartaoValidacaoStrategy, PixValidacaoStrategy, BoletoValidacaoStrategy no pagamento), isolando a regra de negócio específica de cada tipo, por exemplo: o limite máximo do cartão ou o valor mínimo do boleto. Assim, sem que o restante do sistema precise conhecer esses detalhes.
2. **Factory:** PaymentFactory e ValidacaoFactory centralizam a lógica de resolver, a partir da string tipo recebida na requisição, qual implementação concreta de Strategy deve ser usada. Isso evita espalhar lógica de seleção (if (tipo.equals("pix"))...) por controllers e serviços, concentrando esse conhecimento em um único ponto.
3. **Template Method:** a classe abstrata CheckoutTemplate define o esqueleto fixo do fluxo de checkout (authUser >>> processDebit >>> definição de status >>> saveOrderStatus >>> sendReceipt), incluindo o registro das etapas retornadas ao cliente. A subclasse concreta CheckoutProcessor implementa apenas os passos que variam (persistência do pedido e a chamada real à Strategy de pagamento escolhida pela Factory). Isso garante que todo checkout, independentemente do meio de pagamento, passe exatamente pelas mesmas etapas de auditoria, sem risco de alguém "esquecer" um passo ao adicionar um novo fluxo.

Consequências

Positivo:

- Alta coesão: cada Strategy concentra exclusivamente a lógica de um meio de pagamento.
- Baixo acoplamento: o CheckoutTemplate e os controllers dependem apenas das interfaces (PaymentStrategy/ValidacaoStrategy), nunca das implementações concretas.
- Extensibilidade (Open/Closed Principle): adicionar um novo meio de pagamento exige apenas criar uma nova classe Strategy e registrá-la na Factory, sem alterar código já existente e testado.
- Padronização do fluxo: qualquer checkout que reutilize CheckoutTemplate automaticamente segue o mesmo conjunto de etapas de auditoria, sem depender de disciplina manual do desenvolvedor.

Negativo:

- Aumento do número de classes para um MVP relativamente pequeno. Uma Strategy nova por meio de pagamento, além da entrada correspondente na Factory, o que pode parecer verboso para o escopo atual, mas se justifica pelo requisito da disciplina de demonstrar os padrões na prática.
- A Factory depende diretamente da string tipo vinda do corpo da requisição, sem uma validação centralizada da lista de valores aceitos antes da resolução da Strategy: um tipo não cadastrado só é percebido no momento da busca (por exemplo, boleto no ms-checkout, que não tem Strategy registrada e gera erro em tempo de execução em vez de uma resposta de erro mais amigável).
- Como o CheckoutTemplate fixa a ordem das etapas, qualquer meio de pagamento que precise de um fluxo genuinamente diferente (por exemplo, uma etapa extra de confirmação assíncrona) exigiria estender ou reavaliar o Template Method, e não apenas adicionar uma nova Strategy.